Row-Major vs Column-Major Array Traversal in C: An Experimental Analysis of Cache Miss Impact on CPU Performance

Thed Arthur TOKO DIKONGUE
Student at ISTY - Versailles Saint-Quentin-en-Yvelines University
Vélizy-Vllacoublay
thdarthur@gmail.com

***Abstract*—This paper presents an experimental study on the performance impact of memory access patterns in C programs, with a specific focus on two-dimensional array traversal. We demonstrate, through reproducible benchmarks and hardware performance counters, that iterating a matrix with the column index in the outer loop—i.e., `tab[j][i]` with `j` as the outer loop—results in significant performance degradation compared to row-major traversal (`tab[i][j]`, `i` outer). Experiments were conducted on an AMD Ryzen 7 3700U (Zen+, x86-64, Ubuntu Linux) using GCC at `-O2` optimization level. Results from `perf stat` hardware counters show that column-major traversal generates approximately 20× more cache misses than row-major traversal, with a cache miss rate of 29.65% versus 7.60%, translating into a 3.7× slowdown on a 4096×4096 integer matrix (64 MB). Static and dynamic analysis using MAQAO OneView further reveals that the column-major inner loop achieves only 33% vectorization coverage versus 100% for the row-major case, and an Array Access Efficiency of 51.1% versus 100%, confirming that strided memory access prevents effective SIMD utilization. This work provides a concrete, hardware-validated demonstration of spatial locality principles, intended as both a pedagogical reference and a practical guide for performance-sensitive C and C++ development.**

***Index Terms*—cache miss, spatial locality, memory access patterns, row-major order, column-major order, CPU performance, SIMD vectorization, AMD Zen+, hardware performance counters, MAQAO**

I. Introduction

Modern processors achieve high throughput largely through a multi-level cache hierarchy (L1, L2, L3) that bridges the latency gap between the CPU core and main memory (DRAM). The fundamental unit of data transfer between memory and cache is the *cache line*—typically 64 bytes on x86-64 architectures—meaning that even a single-byte read triggers a 64-byte transfer from the next memory level.

Two-dimensional arrays in C are laid out in memory using *row-major order*: elements of the same row are stored at consecutive memory addresses. When a program accesses `tab[i][j]` with `j` varying fastest (the inner loop), the CPU reads consecutive memory locations, exploiting the spatial locality principle. Conversely, when `j` is the outer loop and `i` varies fastest in the inner loop, each successive access is separated by $N \times$ `sizeof(int)` bytes, potentially causing a cache miss on every access once the matrix exceeds the cache size.

This phenomenon is well-known in the computer architecture literature [1], yet concrete, hardware-validated demonstrations combining timing benchmarks, hardware counters, and static analysis tools remain scarce in accessible form. This paper fills that gap by providing:

- A reproducible C benchmark measuring both access patterns across seven matrix sizes from 64×64 to 4096×4096;
- Hardware performance counter measurements via Linux `perf stat`, quantifying cache misses at the L1 and LLC levels;
- Static and dynamic analysis via MAQAO OneView, exposing vectorization degradation and array access efficiency caused by strided memory access;
- A comparative analysis across two compiler optimization levels (`-O0` and `-O2`).

The remainder of this paper is organized as follows. Section II presents theoretical background on cache hierarchy and spatial locality. Section III describes the experimental setup. Section IV presents timing results. Section V analyzes hardware counters. Section VI details MAQAO findings. Section VII discusses implications, and Section VIII concludes.

II. Theoretical Background

A. CPU Cache Hierarchy

Contemporary x86-64 processors implement a multi-level cache hierarchy to mitigate the latency disparity between processing units and DRAM. On the AMD Ryzen 7 3700U used in this study:

- **L1 data cache**: 32 KB per core, latency ≈4 cycles
- **L1 instruction cache**: 64 KB per core
- **L2 cache**: 512 KB per core (private), latency ≈12 cycles
- **L3 cache (LLC)**: 4 MB shared, latency ≈40 cycles
- **Main memory (DRAM)**: 5795 MB, latency ≈200–300 cycles

The ratio of cache hit latency to DRAM latency is typically 50:1 to 100:1. A program generating frequent DRAM accesses due to cache misses therefore experiences severe throughput degradation regardless of computational complexity.

B. Cache Lines and Spatial Locality

When the CPU accesses a memory address not present in any cache level (a *cache miss*), it fetches an entire cache line of 64 bytes—equivalent to 16 consecutive 32-bit integers—from the next level. This mechanism exploits *spatial locality*: the empirical observation that programs tend to access memory addresses proximate to recently accessed ones [2].

For a C array `int tab[N][N]`, the element `tab[i][j]` resides at address:

$$\text{addr}(\texttt{tab[i][j]}) = \text{base} + (i \cdot N + j) \cdot \texttt{sizeof(int)} \quad (1)$$

C. Access Pattern Analysis

Row-major traversal (`j` inner): consecutive values of `j` produce addresses differing by exactly 4 bytes. Accessing `tab[i][0]` through `tab[i][15]` loads 16 integers from a single cache line after one miss—a theoretical miss rate of $1/16 = 6.25\%$.

Column-major traversal (`i` inner): consecutive values of `i` produce addresses differing by $N \times 4$ bytes. For $N = 4096$, this stride equals 16,384 bytes—256 cache lines apart. Each inner loop iteration accesses a cache line unlikely to be resident, producing a miss on virtually every access.

Table I summarizes the theoretical miss rates.

TABLE I: Theoretical Cache Miss Rate Analysis ($N = 4096$, 32-bit integers)

Pattern	Inner	Stride	Miss/N	Miss rate
Row-major	`j`	4 B	1	∼6.25%
Column-major	`i`	16,384 B	16	∼100%

III. Experimental Setup and Environment

A. Hardware Environment

The experiments were conducted on a machine equipped with an AMD Ryzen 7 3700U processor (Zen+ microarchitecture, 14nm, 4 physical cores / 8 threads via SMT). Fig. 1 shows the full memory topology as reported by `lstopo`.

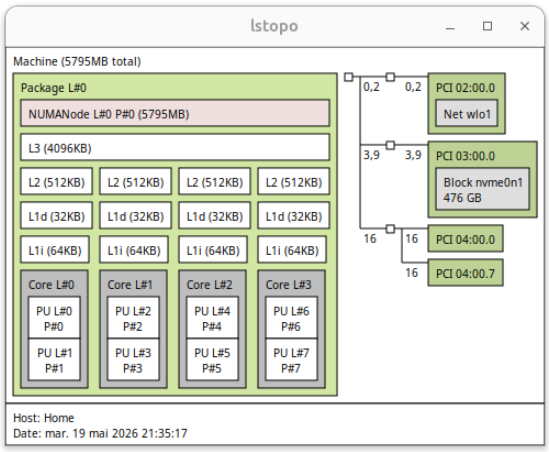


Fig. 1: Hardware memory topology (lstopo): AMD Ryzen 7 3700U, 4 cores, L1d 32 KB, L2 512 KB per core, L3 4 MB shared, 5795 MB RAM.

Each core has a private 32 KB L1 data cache, a 64 KB L1 instruction cache, and a private 512 KB L2 cache. All four cores share a single 4 MB L3 (LLC). The cache line size is 64 bytes (`clflush size: 64` in `/proc/cpuinfo`). Total system RAM is 5795 MB on a single NUMA node.

TABLE II: Experimental Environment Summary

Component	Specification
CPU	AMD Ryzen 7 3700U (Zen+, 4C/8T, 2.3 GHz)
L1d / L1i	32 KB / 64 KB per core
L2	512 KB per core (private)
L3 (LLC)	4 MB shared
Cache line	64 bytes
RAM	5795 MB (single NUMA node)
OS	Ubuntu Linux (kernel 6.x)
Compiler	GCC (`-O0` and `-O2`)
Profiler	Linux `perf` v6.17.13
Analyzer	MAQAO OneView
Timing	`clock_gettime(CLOCK_MONOTONIC)`
Runs / size	5 (mean reported)

B. Benchmark Implementation

Two traversal functions were implemented to isolate the effect of loop ordering. Both apply the `sin()` function to each element, preventing dead code elimination while introducing a realistic floating-point workload.

```c
void row_comp(int *mat, double *res, int size) {
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            res[i*size + j] = sin(mat[i*size + j]);
}
```

Listing 1: Row-major traversal (cache-friendly)

```c
void column_comp(int *mat, double *res, int size) {
    for (int j = 0; j < size; j++)
        for (int i = 0; i < size; i++)
            res[i*size + j] = sin(mat[i*size + j]);
}
```

Listing 2: Column-major traversal (cache-unfriendly)

Matrices are heap-allocated via `malloc`. Values are initialized with `rand() % 10`. The only structural difference between the two functions is the order of the outer and inner loop indices.

C. Matrix Sizes Tested

TABLE III: Matrix Sizes and Cache Pressure

N	Size	Cache pressure
64	16 KB	Fits entirely in L1d
128	62 KB	Fits in L2
256	256 KB	At L2 boundary
512	1 MB	Between L2 and L3
1024	4 MB	At LLC boundary
2048	16 MB	Exceeds LLC
4096	64 MB	Full RAM access required

IV. Benchmark Results

A. Execution Time

Fig. 2 shows mean execution time as a function of matrix size for both access patterns and both optimization levels.

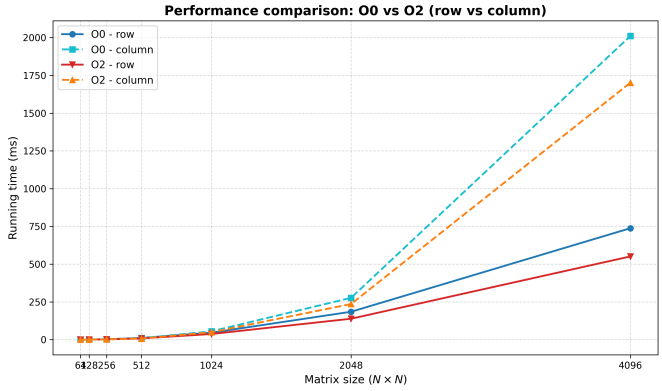


Fig. 2: Execution time (ms) vs. matrix size N: row-major vs. column-major, `-O0` and `-O2` (AMD Ryzen 7 3700U, 5 runs each).

The divergence between access patterns begins at $N = 1024$ (4 MB, at the LLC boundary) and increases monotonically. At $N = 4096$, the column-major pattern is **3.1× slower** under `-O2` and **2.6× slower** under `-O0`. Fig. 3 shows the slowdown ratio explicitly.

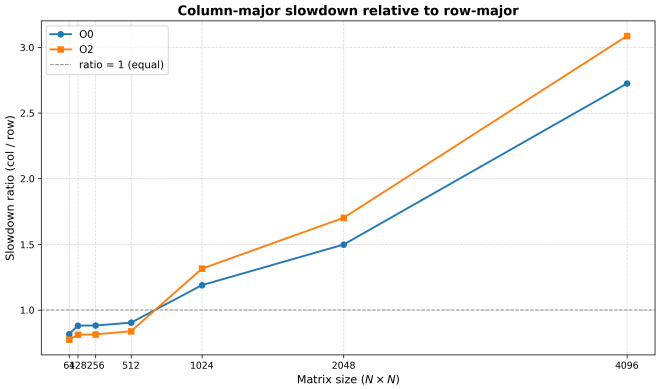


Fig. 3: Column-major slowdown ratio relative to row-major. The cache saturation threshold at $N = 1024$ (LLC = 4 MB) is clearly visible.

B. Raw Timing Data

TABLE IV: Mean Execution Times, `-O2` (ms), AMD Ryzen 7 3700U

N	Size	Row (ms)	Col (ms)	Ratio
64	16 KB	0.140	0.111	0.79
128	62 KB	0.578	0.472	0.82
256	256 KB	2.121	1.728	0.81
512	1 MB	8.368	7.017	0.84
1024	4 MB	37.46	49.25	1.31
2048	16 MB	138.6	235.9	1.70
4096	64 MB	551.1	1701.2	3.09

V. Hardware Performance Counter Analysis

Linux `perf stat` was used to collect hardware performance counters for both patterns on a 4096×4096 matrix under `-O2`. Separate binaries were used per pattern to isolate readings.

TABLE V: `perf stat` Results, $N = 4096$, `-O2`

Counter	row_comp	column_comp
Cache misses	1,904,790	37,279,590
Cache references	25,057,659	125,749,082
L1 load misses	9,290,356	40,823,961
Miss rate	7.60%	29.65%
Elapsed time	552 ms	2034 ms
Miss ratio (col/row)	**≈20×**	

Column-major traversal generates **20× more cache misses** than row-major traversal. The L1 load miss count increases by a factor of 4.4, and the cache miss rate rises from 7.60% to 29.65%, confirming that the CPU fetches data from higher memory levels on nearly one in three accesses. The elapsed time ratio (3.7×) is lower than the miss ratio (20×) because the `sin()` computation partially hides memory latency through out-of-order execution in the row-major case.

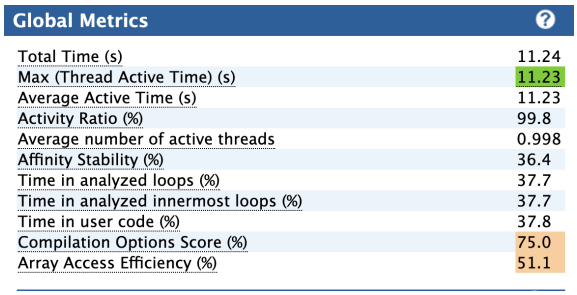
VI. MAQAO Static and Dynamic Analysis

A. Methodology

MAQAO OneView was used to perform combined static (CQA) and dynamic (LProf) analysis [4]. Two separate runs were conducted—one per access pattern—to obtain isolated per-function metrics.

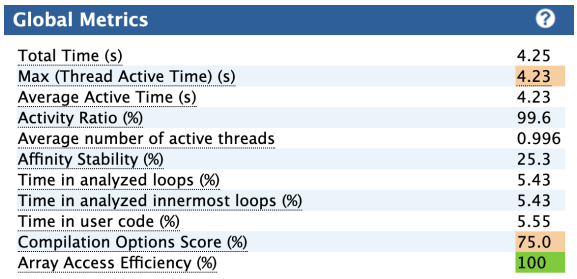
B. Global Metrics

Fig. 4 and Fig. 5 show the Global Metrics panels for `column_comp` and `row_comp` respectively.



Global Metrics	
Total Time (s)	11.24
Max (Thread Active Time) (s)	11.23
Average Active Time (s)	11.23
Activity Ratio (%)	99.8
Average number of active threads	0.998
Affinity Stability (%)	36.4
Time in analyzed loops (%)	37.7
Time in analyzed innermost loops (%)	37.7
Time in user code (%)	37.8
Compilation Options Score (%)	75.0
Array Access Efficiency (%)	51.1

Fig. 4: MAQAO Global Metrics — `column_comp`: Total Time 11.24 s, Array Access Efficiency **51.1%**.



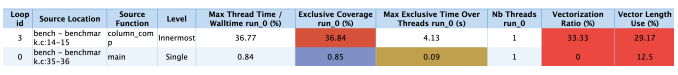
Global Metrics	
Total Time (s)	4.25
Max (Thread Active Time) (s)	4.23
Average Active Time (s)	4.23
Activity Ratio (%)	99.6
Average number of active threads	0.996
Affinity Stability (%)	25.3
Time in analyzed loops (%)	5.43
Time in analyzed innermost loops (%)	5.43
Time in user code (%)	5.55
Compilation Options Score (%)	75.0
Array Access Efficiency (%)	100

Fig. 5: MAQAO Global Metrics — `row_comp`: Total Time 4.25 s, Array Access Efficiency **100%**.

The *Array Access Efficiency* metric directly quantifies memory access quality: MAQAO reports **100%** for `row_comp` and **51.1%** for `column_comp`, confirming that nearly half of all memory bandwidth is wasted in the column-major case.

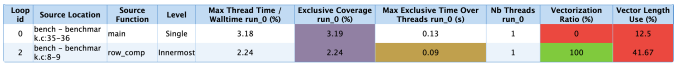
C. Expert Summary — Loop Analysis

Fig. 6 and Fig. 7 show the Expert Summary loop tables for each pattern.



Loop Id	Source Location	Source Function	Level	Max Thread Time / Walltime run_0 (%)	Exclusive Coverage run_0 (%)	Max Exclusive Time Over Threads run_0 (s)	Nb Threads run_0	Vectorization Ratio (%)	Vector Length Use (%)
3	bench – benchmark.c:14–15	column_comp	Innermost	36.77	36.84	4.13	1	33.33	29.17
0	bench – benchmark.c:35–36	main	Single	0.84	0.85	0.09	1	0	12.5

Fig. 6: MAQAO Expert Summary — `column_comp` (Loop 3, benchmark.c:14–15): 36.84% app. coverage, 33.33% vectorization.



Loop Id	Source Location	Source Function	Level	Max Thread Time / Walltime run_0 (%)	Exclusive Coverage run_0 (%)	Max Exclusive Time Over Threads run_0 (s)	Nb Threads run_0	Vectorization Ratio (%)	Vector Length Use (%)
0	bench – benchmark.c:35–36	main	Single	3.18	3.19	0.13	1	0	12.5
2	bench – benchmark.c:8–9	row_comp	Innermost	2.24	2.24	0.09	1	100	41.67

Fig. 7: MAQAO Expert Summary — `row_comp` (Loop 2, benchmark.c:8–9): 2.24% app. coverage, 100% vectorization.

D. Quantitative Comparison

TABLE VI: MAQAO OneView Metrics Comparison, $N = 4096$, `-O2`

Metric	row_comp	column_comp
Coverage (% app. time)	2.24%	36.84%
Vectorization ratio (%)	100%	33.33%
Vector length use (%)	41.67%	29.17%
Array Access Efficiency (%)	**100**	**51.1**
Total time (s)	4.25	11.24
CQA speedup if fully vectorized	2.93×	7.71×

The coverage metric is the most striking result: `column_comp` consumes **36.84%** of total application time versus **2.24%** for `row_comp`—a 16× difference directly attributable to cache miss stalls. The vectorization ratio drops from 100% to 33%: GCC generates packed AVX instructions (`VMOVUPD`, `VINSERTF128`) for the row-major loop but cannot apply these to non-contiguous strided accesses. This explains the potential speedup of 7.71× if fully vectorized—the column-major loop leaves substantial SIMD performance unrealized.

VII. Discussion

A. Cache Saturation Threshold

Results identify a clear *cache saturation threshold* at $N = 1024$ (4 MB), coinciding exactly with the LLC size of the Ryzen 7 3700U. Below this threshold, the Zen+ hardware prefetcher compensates for the strided access pattern—it detects the regular stride and preloads cache lines proactively, making column-major paradoxically faster. Above the LLC boundary, the prefetcher cannot keep up with demand and cache miss penalties dominate.

B. Compiler Behavior

Even under `-O2`, GCC does not automatically perform loop interchange to transform column-major access into row-major order. This transformation is only legal when loop iterations are provably independent. The programmer bears primary responsibility for access pattern design.

C. Compounding Effect: Vectorization Loss

Beyond cache misses, the MAQAO analysis reveals that the strided access pattern also degrades SIMD vectorization: the vectorization ratio drops from 100% to 33%, and Array Access Efficiency from 100% to 51.1%. This compounds the performance penalty beyond what cache miss counts alone would predict.

D. Practical Implications

- Always iterate C arrays in row-major order (`j` as inner loop);
- When column access is required by algorithm logic, consider transposing the matrix before the hot loop;
- The penalty is negligible for matrices fitting in L2 but severe beyond the LLC boundary;
- Modern prefetchers (Zen+, Apple M-series) partially compensate for small working sets but cannot eliminate the penalty at scale.

VIII. Conclusion

This paper provided a hardware-validated, end-to-end experimental demonstration of cache miss impact induced by column-major array traversal in C. Using timing benchmarks, Linux `perf` hardware counters, and MAQAO static/dynamic analysis on an AMD Ryzen 7 3700U, we showed that:

- Column-major traversal generates **20× more cache misses** than row-major for a 4096×4096 matrix;
- This translates into a **3.7× execution time slowdown**;
- Vectorization ratio drops from 100% to 33%, and Array Access Efficiency from 100% to 51.1%, revealing a compounding degradation;
- A clear cache saturation threshold exists at the LLC boundary (4 MB), below which the hardware prefetcher masks the effect.

Future work could extend this study to multi-threaded contexts (OpenMP), GPU memory coalescing (CUDA), tiled matrix traversal as mitigation, and comparisons across multiple microarchitectures (Intel Alder Lake, ARM Cortex-A).

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[2] P. J. Denning, "The locality principle," *Communications of the ACM*, vol. 48, no. 7, pp. 19–24, Jul. 2005.

[3] U. Drepper, "What every programmer should know about memory," *Red Hat, Inc.*, Tech. Rep., 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[4] D. Barthou *et al.*, "MAQAO: Modular assembler quality analyzer and optimizer for Itanium-2," in *Proc. Int. Conf. on High Performance Computing*, 2005.

[5] A. Fog, "Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms," Tech. Rep., 2023. [Online]. Available: https://www.agner.org/optimize/